

CLI Commands Handbook

Everything you need to build, run, and debug NexConvo from the command line — frontend, .NET microservices, Docker infrastructure, database migrations, git, and troubleshooting.

Next.js 15 · port 3003

.NET 9 microservices

YARP Gateway

PostgreSQL · Redis · RabbitMQ · Seq

Docker Compose

Repo: `d:\Resources\Projects\NexConvo` • Shell: PowerShell / Git Bash • Run commands from the repo root unless a `cd` is shown.

0 Prerequisites

install once

TOOL	VERSION	CHECK
Node.js + npm	18 LTS or newer	<code>node -v npm -v</code>
.NET SDK	9.0.313 (pinned in <code>global.json</code>)	<code>dotnet --version</code>
Docker Desktop	latest (with Compose v2)	<code>docker --version docker compose version</code>
EF Core CLI	for migrations	<code>dotnet tool install --global dotnet-ef</code>
Git	any recent	<code>git --version</code>

1 Quick Start — Run the Whole Stack

first run, in order

Each long-running service needs its **own terminal**. Start infrastructure first, then Identity (others depend on its JWT), then the gateway, then the rest, then the frontend.

1 Start infrastructure (Postgres, Redis, RabbitMQ, Seq)

```
# from repo root - copy env first, then start containers
cp .env.example .env # set POSTGRES/REDIS/RABBITMQ passwords inside
docker compose up -d
```

2 Apply database migrations (Identity first)

```
dotnet ef database update `
--project src/services/Identity/NexConvo.Identity.Infrastructure `
--startup-project src/services/Identity/NexConvo.Identity.Api
```

3 Run the Identity service (required before others)

```
dotnet run --project src/services/Identity/NexConvo.Identity.Api --launch-profile http
```

4 Run the API Gateway (YARP)

```
dotnet run --project src/gateway/NexConvo.Gateway --launch-profile http # → http://localhost:5055
```

5 Run any other services you need (each in its own terminal)

```
dotnet run --project src/services/CoreCrm/NexConvo.CoreCrm.Api --launch-profile http
```

6 Run the frontend

```
cd frontend
npm install # first time only
npm run dev # → http://localhost:3003
```

Fast path (frontend UI only): the landing, login, and dashboard pages are static — you can skip steps 1–5 and just run step 6.

```
cd frontend
cp .env.example .env.local      # set API_GATEWAY_URL (server-only)

npm install                     # install dependencies
npm run dev                     # dev server → http://localhost:3003
npm run build                   # production build (NOTE: "run build", not "npm build")
npm start                       # serve the production build → :3003
npm run lint                    # ESLint (bans any / non-null !)
npm run typecheck               # tsc --noEmit (strict)
npm run test                    # Vitest (run once)
npm run test:watch              # Vitest watch mode
```

PAGE	URL
Landing	http://localhost:3003/
Login (split-screen)	http://localhost:3003/login
Dashboard shell	http://localhost:3003/dashboard

Toggle **theme** (light/dark) and **language** (English ↔ বাংলা) from the top-right of any page.

Compose lifecycle — the local stack (Postgres, Redis, RabbitMQ, Seq; gateway is opt-in).

```
cp .env.example .env           # required: sets DB/Redis/RabbitMQ passwords
docker compose up -d           # start infra only (detached)
docker compose up -d --profile apps  # infra + ALL services + gateway (containers)
docker compose ps              # list running services + health
docker compose logs -f         # tail all logs
docker compose logs -f postgres  # tail one service
docker compose restart rabbitmq  # restart one service
docker compose stop            # stop (keep containers + data)
docker compose down            # stop + remove containers (keeps volumes)
docker compose down -v         # + wipe volumes (clean DB / fresh start)
docker compose config          # validate the compose file
docker compose pull            # pull newer base images
```

Run the whole backend as containers (the professional inner-loop — infra + all 6 services + gateway).

```
docker compose --profile apps up -d --build  # build + start everything as containers
docker compose --profile apps ps            # status of the full stack
docker compose --profile apps logs -f identity  # tail one service

# Debug ONE service locally while the rest run as containers:
docker compose --profile apps up -d --scale chat=0  # drop chat's container...
# ...then press F5 on "Chat API" in VS Code (infra + other services stay containerized)

docker compose --profile apps down          # stop + remove app containers + infra
docker compose stop                         # just pause everything (keep data)
```

Host ports mirror each service's launchSettings (Identity :5047, CRM :5081, ...) so a container and a local `dotnet run` are reachable the same way. The frontend talks only to the gateway (`:8080`).

Get a shell inside the running infra (uses service names — project-independent).

```
# PostgreSQL - connect to a specific service DB
docker compose exec postgres psql -U nexconvo -d nexconvo_identity
docker compose exec postgres psql -U nexconvo -c "\l"  # list all databases

# Redis (password-protected via --requirepass)
docker compose exec redis redis-cli -a "$env:REDIS_PASSWORD" ping

# RabbitMQ
docker compose exec rabbitmq rabbitmqctl status
docker compose exec rabbitmq rabbitmqctl list_queues
```

General Docker — containers, images, volumes, cleanup & debugging.

```
docker ps          # running containers      docker ps -a      # all (incl. stopped)
docker images      # local images            docker stats      # live CPU/mem
docker logs -f <container>  # follow a container's logs
docker exec -it <container> sh  # shell into any container
docker stop <c>      docker start <c>      docker rm <c>      # manage a container
docker volume ls     docker network ls     # volumes / networks
docker system df      # disk usage by Docker
docker builder prune  # clear build cache
docker system prune -af + docker volume prune  # reclaim space
```

[DESTRUCTIVE] `docker compose down -v`, `system prune -af`, and `volume prune` delete data/volumes. Your databases live in the `postgres_data` volume — pruning it wipes all local data.

Gateway port — two cases: Docker-composed gateway (`--profile apps`) → `http://localhost:8080` (this is what the frontend `.env.example` points to). Gateway via `dotnet run` → `http://localhost:5055`. Set the frontend's `API_GATEWAY_URL` to match whichever you run.

4 .NET Backend — Build, Run, Test

SDK 9.0.313 · net9.0

```
dotnet restore           # restore NuGet packages (solution-wide)
dotnet build             # build the whole solution (Debug)
dotnet build -c Release  # release build
dotnet test              # run all test projects under tests/
dotnet test tests/NexConvo.Identity.Tests # a single test project
dotnet format            # auto-format to .editorconfig
dotnet clean             # remove bin/ and obj/
```

Run a single service — `dotnet run --project <path> --launch-profile http`:

SERVICE	PROJECT PATH	HTTP PORT
Gateway (YARP)	src/gateway/NexConvo.Gateway	5055
Identity start 1st	src/services/Identity/NexConvo.Identity.Api	5047
Core CRM	src/services/CoreCrm/NexConvo.CoreCrm.Api	5081
Chat	src/services/Chat/NexConvo.Chat.Api	5132
Voice	src/services/Voice/NexConvo.Voice.Api	5022
AI Assistant	src/services/AiAssistant/NexConvo.AiAssistant.Api	5124
Automation Worker	src/services/Automation/NexConvo.Automation.Worker	5228

5 EF Core — Database Migrations

per service

Pattern: `--project` = the service's **Infrastructure** project, `--startup-project` = its **Api**. Example uses CoreCrm.

```
# apply latest migrations to the DB
dotnet ef database update `
  --project src/services/CoreCrm/NexConvo.CoreCrm.Infrastructure `
  --startup-project src/services/CoreCrm/NexConvo.CoreCrm.Api

# create a new migration
dotnet ef migrations add AddLeadScore `
  --project src/services/CoreCrm/NexConvo.CoreCrm.Infrastructure `
  --startup-project src/services/CoreCrm/NexConvo.CoreCrm.Api

# revert to a previous migration
dotnet ef database update <PreviousMigrationName> --project ... Infrastructure --startup-project ... Api
```

On first `docker compose up`, Postgres auto-runs `infra/postgres/init/01-databases.sql`, creating the six service databases: `nexconvo_identity`, `nexconvo_crm`, `nexconvo_chat`, `nexconvo_voice`, `nexconvo_automation`, `nexconvo_ai`.

APP / SERVICE	PORT	INFRASTRUCTURE	PORT / URL
Frontend (Next.js)	3003	PostgreSQL	5432
Gateway — Docker	8080	Redis	6379
Gateway — dotnet run	5055	RabbitMQ (AMQP)	5672
Identity	5047	RabbitMQ (UI)	localhost:15672
Core CRM	5081	Seq (logs UI)	localhost:5341
Chat	5132		
Voice	5022		
AI Assistant	5124		
Automation Worker	5228		

```
git status           # what changed
git branch           # list branches (current: feat/identity-auth)
git checkout -b feat/my-feature # create + switch to a new branch
git add .            # stage all changes
git commit -m "feat(web): add landing page"
git push -u origin feat/my-feature # push + set upstream
git pull             # fetch + merge latest
git log --oneline -10 # recent history
git switch main      # back to main
```

SYMPTOM	FIX
Port already in use (e.g. 3003 / 5047)	Find & kill: <code>netstat -ano findstr :3003 → taskkill /PID <pid> /F</code> . Or change the frontend port in <code>frontend/package.json</code> .
Docker errors / "cannot connect to daemon"	Start Docker Desktop first. Verify with <code>docker ps</code> . Re-run <code>docker compose up -d</code> .
Compose: "set POSTGRES_PASSWORD in .env"	You skipped the env copy: <code>cp .env.example .env</code> and fill in the passwords.
Frontend can't reach the API	Make <code>frontend/.env.local</code> → <code>API_GATEWAY_URL</code> match the gateway you ran (:8080 Docker / :5055 dotnet run).
NuGet / build weirdness	<code>dotnet clean → dotnet restore → dotnet build</code> .
node_modules broken	<code>cd frontend → delete node_modules + package-lock.json → npm install</code> .
DB schema out of date	Re-run EF database <code>update</code> (§5), or full reset: <code>docker compose down -v</code> then <code>up -d</code> + migrate.
Seq (logs) container keeps exiting	Needs <code>SEQ_FIRSTRUN_NOAUTHENTICATION=true</code> (already set in compose). If it still fails on <code>/data/Seq.json</code> , the old volume is stale: <code>docker compose up -d --force-recreate --renew-anon-volumes seq</code> .